



Group: 7

Board Member: Mario Campanelli

**Project Title: Exploring Foundational Models
for Strong Interactions**

Word Count:2983

Identifier	Roles, Tasks, etc.
AA	Communications Officer, OmniJet- α Research Lead
BB	Minute Taker, Background Research Lead
CC	OmniLearn Research Lead, Testing Assistance
DD	Literature Coordinator, Machine Learning Research Lead
EE	Coding Lead, General Editor

Abstract

Foundation models are emerging tools in jet physics, capable of performing multiple tasks with improved performance where data or computational resources are limited. While foundation models are mostly compared to prior tools, we examine and compare two foundation models, OmniJet- α and OmniLearn. We assess their architectures, compare performance, and identify causes of performance gaps to guide future model development. We demonstrate that both are viable foundation models, capable of jet tagging and generation, with enhanced performance from transfer learning. OmniLearn outperforms as a jet tagger, but this advantage disappears when its input features match OmniJet- α . Discrete tokenisation in OmniJet- α has negligible impact, while OmniLearn’s PET architecture offers benefits when training on small datasets. We conclude that OmniLearn’s flexibility is the key differentiator in their architectures’ suitability for future models, while the dimensionality of particle representations is the main determinant of peak performance regardless of architecture.

Contents

1	Introduction and Theory
1.1	Particle Jets {BB}
1.2	Foundational Models for Jet Physics {DD}
1.3	OmniJet- α {AA}
1.4	OmniLearn {CC}
2	Methodology
2.1	Hypotheses {EE}
2.2	Evaluation Framework for Foundation Model Performance {EE}
2.3	Experimental Design and Testing Procedure {EE}
3	Results
3.1	OmniJet- α
3.2	OmniLearn
3.3	Comparative Results
4	Discussion {AA, EE}
5	Conclusion {AA, BB, CC, EE}

References

14

2	1 Introduction and Theory
2	
3	1.1 Particle Jets {BB}
3	Particle colliders are centres of discovery for
4	high-energy physics (HEP), enabling the dis-
4	covery of [1] new particles and [2] uncover-
4	ing anomalies that may reveal new physics [3].
4	High-energy collisions create massive and highly
4	Lorentz-boosted particles [4]. Non-leptonic
4	products, however, are rarely detected directly
5	[5], as colour confinement and the decay of unsta-
5	ble heavy particles creates streams of less mas-
6	sive particles. These particles form cones, closely
6	aligned to the direction of the initial collision
6	product, called jets.
6	Jets must be understood to reconstruct col-
6	lider processes. This involves tasks including
12	jet tagging, generation, and anomaly detection.
13	Jet tagging classifies jets by deciphering their
13	origin from the observed final products. Acc-
13	urate tagging enables the identification of in-

interactions involving massive or rare particles [6], and the calculation of interaction cross-sections. Generation synthesizes jets by creating sets of particles that accurately reflect real observations [7]. This enables cheap testing of predictions and refinement of experimental design. Finally, anomaly detection flags events that don't fit predictions and may imply new physics [Belis2024AnomalyML].

1.2 Foundational Models for Jet Physics {DD}

Foundation models are a class of machine learning (ML) models that perform multiple tasks. Large language models (LLMs) are the most prevalent example, training across tasks including text generation, summarisation, and answering questions. By sharing tasks built around some fundamental understanding [8], which is language structures in the case of LLMs, foundation models can be flexible and often achieve higher performance on limited training compared to specialised models [9].

Foundation models for jet physics similarly generalise across tasks. HEP foundation models developed to date use pretraining on a single task with a large available dataset to learn general representations of jet structure. For example, generating jets may teach a model the physical relationships between jet constituents that underpin tagging. Pretraining produces a backbone, a neural network whose output feeds into heads, smaller neural networks that are cheaper to train, and adapt the model to specific tasks [10]. This process of transfer learning, also called fine-tuning, makes the backbone reusable [11].

Foundation models for HEP build on deep learning models previously used for individual tasks, which in turn improved on previous non-

ML approaches. For example, early jet tagging methods relied on calculating overall substructure variables like jet mass or N-subjettiness – which measures how well a jet is represented by N subjects [12] – with cutoffs of each variable for different jet types. Deep learning models improved on these methods by considering the individual particles (jet constituents) and their properties [13], raising performance with the volume of input information [14]. These deep learning models are, however, computationally expensive to train, and tagging models rely on large datasets of accurately-labelled jets which are expensive to produce. They are also trained for specific tasks, datasets, particle types, and detector geometries [15]. Foundation models therefore promise greater flexibility, improved performance on tasks where data is scarce, and lower computational demands.

1.3 OmniJet- α {AA}

OmniJet- α is the first foundation model for jet physics [16]. Inspired by LLMs like Chat-GPT, it demonstrates that a backbone pretrained on generative tasks can benefit jet tagging. It uses JetClass for training and analysis, a benchmark dataset of simulated jets across ten classes used frequently in HEP.

OmniJet- α takes the kinematic features of each jet constituent – specifically p_T (transverse momentum), η_{rel} (pseudorapidity – a proxy for the polar angle from the jet direction) and ϕ_{rel} (azimuthal angle) – and uses a Vector Quantized Variational Encoder (VQ-VAE) to compress these into discrete particle tokens. The VQ-VAE neural network thus maps a continuous 3D feature space into a codebook of allowed integer tokens. The generative pretraining objective requires a GPT-style autoregressive transformer

to predict a sequence of tokens, using the prior sequence as input. Jets are naturally unordered, variable-length particle sets, but OmniJet- α orders them by descending p_T to allow the autoregressive sequence modelling. Thus, the backbone “learns” jet structure and correlations over millions of iterations. Jet tagging is achieved by a classifier head which considers an entire sequence, and appends a class-token encoding the jet origin.

1.4 OmniLearn {CC}

OmniLearn deviates from the LLM-inspired paradigm [15], utilising a point-edge-transformer (PET) powered model that considers the features of all jet constituents as continuous variables. For classification, the entire jet is taken as an input, with a class-token generated in a similar manner to OmniJet- α . It can then be fine-tuned onto a wider range of tasks: jet generation, anomaly detection, likelihood ratio estimation, and tagging on varied detectors, jet types, and datasets. These downstream tasks align with the learned representation, considering all jet constituents simultaneously rather than sequentially. For example, OmniLearn utilises a diffusion generative model where particle features are randomised, then refined simultaneously via multiple transformations to synthesize a valid jet.

OmniLearn’s introductory paper proposes that jet physics can uniquely utilise high-quality simulations to cost-effectively produce labelled datasets. As such, it uses the JetClass database for supervised classification pretraining. It is then fine-tuned onto real datasets for all tasks.

Each particle in OmniLearn’s representation has the same kinematic features as in OmniJet- α , as well as particle identification data (charge

and particle type), and its energy and momentum as a proportion of that of the jet. This increases the available input information, potentially extending the key advantages of deep learning models [17].

2 Methodology

2.1 Hypotheses {EE}

Both models’ introductory papers indicated they would display foundation-model capability. We also hypothesised that OmniLearn would outperform OmniJet- α as a foundation model. This advantage could arise from the volume of OmniLearn’s input information, its supervised pretraining, or OmniJet- α ’s imposed sequential learning and tokenization losing information about continuous unordered jets.

2.2 Evaluation Framework for Foundation Model Performance {EE}

To investigate the efficacy of OmniJet- α and OmniLearn as foundation models, it was necessary to show they perform multiple tasks and benefit from transfer learning. As OmniJet- α is only capable of generation and tagging, these tasks were the basis of investigation. Classifier transfer learning is the most appropriate comparative test, being the only task for which OmniJet- α uses transfer learning.

The models were run using code provided in GitHub repositories accompanying their respective papers. Jobs were run on the UCL DIAS cluster, providing CPU, GPU, and storage resources. Dr. Edmondson and Yuanda Zhu were instrumental in providing access and assisting with the creation of Singularity environments

containing the necessary dependencies for each model.

Classifier training was evaluated through the metrics of accuracy and the area (AUC) under the receiver operator characteristic curve (ROC). Accuracy measures the proportion of jets correctly tagged; however, a dataset imbalance can make accuracy a misleading metric if the classifier is biased towards the majority jet type. The ROC curve maps the true positive rate against the false positive rate for different classification thresholds [18, 19]. AUC is the best holistic performance measure for binary classifiers, and is consistently applied when comparing deep learning jet taggers.

We quantified our generative results using Wasserstein distance, a stable, commonly used metric that measures the mass movement needed to equate two distributions [20, 21].

All uncertainties presented are standard errors calculated from five repeat tests with random seeding.

2.3 Experimental Design and Testing Procedure {EE}

OmniLearn is limited to classifying two jet types, fulfilling the predominant demand for binary jet taggers [6, 15, 16, 22]. Consequently, our classifier tests focused on top-tagging versus a quark/gluon (QCD) background, a benchmark across HEP deep learning tests HEP [23]. We also use top-jet feature distributions for generative testing.

To evaluate the advantages of transfer learning, the models were fine-tuned on datasets ranging from 10^2 to 10^6 jets each of the signal and quark/gluon types. This upper bound aligns with the size of most labelled datasets of real jets [24], and the point beyond which most mod-

els see marginal performance increases [25]. In each test, the best iteration across ten training epochs was selected based on its performance on a separate validation dataset, mitigating overfitting. Standardising the training steps and sample size between models controls two key factors that influence performance [26].

This test was repeated with pretrained backbones provided in each repository (which we call "ideal"), control backbones pretrained on half of the JetClass database once in fifty epochs of 10^6 jets, and the models trained from scratch. We further tested the influence of transfer learning by allowing the parameters of the OmniJet- α control backbone to either adapt or remain fixed during fine-tuning {DD}.

Convergence speeds of the two models were tested to assess their suitability for research with limited computational resources. This is achieved by varying the number of training epochs on a 10^4 jet dataset.

Finally, classifier performance using the control backbone was evaluated on five jet types from the JetClass dataset, using 10^6 each of signal and quark/gluon jets for ten training epochs to test each backbone's generalisability.

For generative testing, both models produced samples of 10^4 top jets for comparison to the particle features and distribution of ten-thousand simulated jets. OmniJet- α is trained and evaluated on the JetClass database, while OmniLearn uses the JetNet database for generative training. Direct comparisons cannot therefore be made between these results.

Two potential causes for performance discrepancies between the models were investigated further. We fine-tuned OmniLearn onto a modified dataset containing only the kinematic constituent features used by OmniJet- α to test the impact of particle input information. We also

varied the size of OmniJet- α 's codebook to quantify whether tokenization introduces information loss that affects downstream classification performance {AA}. This was not measured in the original paper.

3 Results

3.1 OmniJet- α

Figure 1.1 clearly demonstrates that the generative pretraining improves tagging performance through transfer learning. This advantage is strongest on small datasets, but remains significant at the largest size, implying it provides an improvement in maximum performance as well as a headstart on learning. Figures 1.2 and 1.3 show that flexible backbones outperform the AUC of the fixed backbone for all dataset sizes, and converge to peak performance more quickly. Fine-tuning a larger number of parameters (not just those in the task-specific head) therefore improves performance, indicating that the jet representations the backbone learns in pretraining are, unsurprisingly, not perfectly suited for the classification task.

Figure 2.0 demonstrates that OmniJet- α is effective as a generative model, producing reasonable agreement with the JetClass data. The mean Wasserstein distance for the particle-feature distributions was $(2.1 \pm 0.1) \times 10^{-2}$. The Wasserstein distance between jet mass distributions was $(1.51 \pm 0.03) \times 10^{-3}$.

3.2 OmniLearn

OmniLearn was also an effective transfer learning model, with both pretrained backbones outperforming from-scratch training in top-tagging, as demonstrated in Figure 3.1. Figure 3.2

demonstrates that removing additional input particle information to align with OmniJet- α significantly reduces OmniLearn's performance on top-tagging, plateauing at a lower maximum.

Figure 3.3 demonstrates that OmniLearn is also effective as a generative model, producing reasonable agreement with the JetNet data on jet-level features. Producing particle-level feature plots is a key next step beyond this report. The mean Wasserstein distance for the particle-feature distributions was $(3.03 \pm 0.09) \times 10^{-2}$. The Wasserstein distance between jet mass distributions was $(3.99 \pm 0.05) \times 10^{-2}$.

3.3 Comparative Results

Figures 4.1-4.3 clearly demonstrate that OmniLearn outperforms OmniJet- α on top-tagging across all dataset sizes, backbones, and in from-scratch training. Tables 1-2 reveal that this advantage generalises across a variety of jet types.

Figure 4.4 indicates, however, that the performance difference on datasets of 10^3 jets or larger is negligible once the additional particle information is removed from OmniLearn. Therefore, the particle inputs seem to be the key driver of peak performance. There remains a significant performance gap with smaller datasets. Figures 4.3 and 4.4 show that OmniLearn's from-scratch training outperforms the pretrained OmniJet- α in the low data regime and imply that fundamental elements of OmniLearn's architecture, as opposed to its pretraining, are producing the performance advantages here.

Figure 4.5 indicates that the models are essentially identical in their computational needs, converging to near-maximum performance within seven epochs of training, with OmniLearn having a slight advantage between three and five epochs. This also indicates the ten

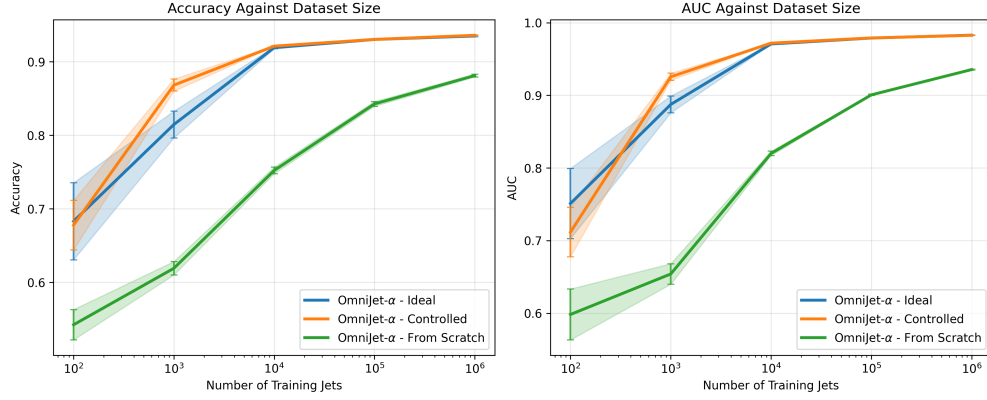


Figure 1.1: OmniJet- α shows a top-tagging advantage in transfer learning over from-scratch training {BB, DD, EE}.

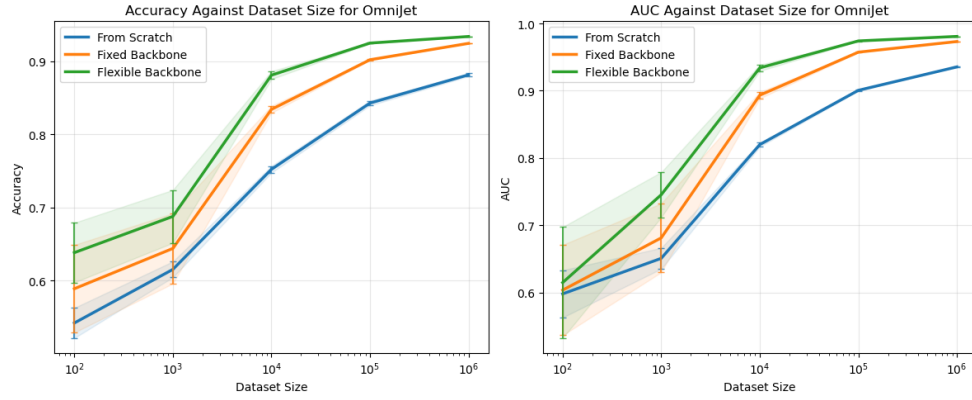


Figure 1.2: Top-tagging efficiency of OmniJet- α for three training strategies {DD}.

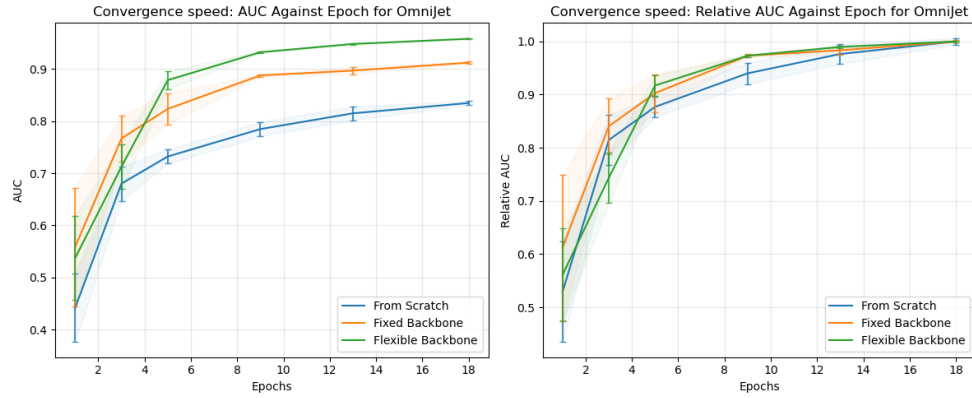


Figure 1.3: Convergence speed of OmniJet- α for three training strategies {DD}.

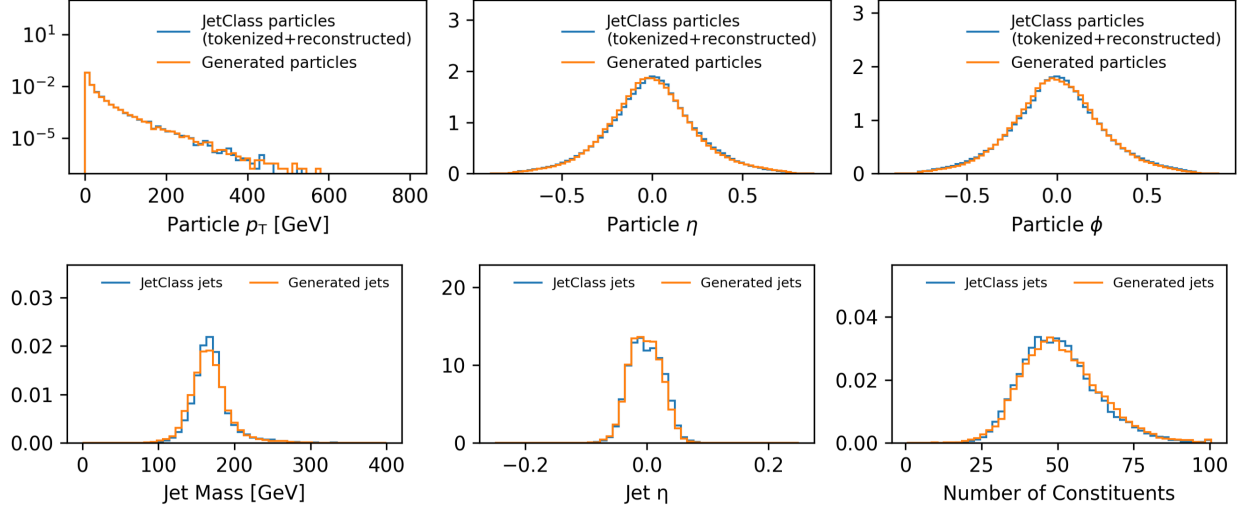


Figure 2.0: OmniJet- α generated against JetClass normalised distributions. Top: particle features. Bottom: jet features {AA, EE}.

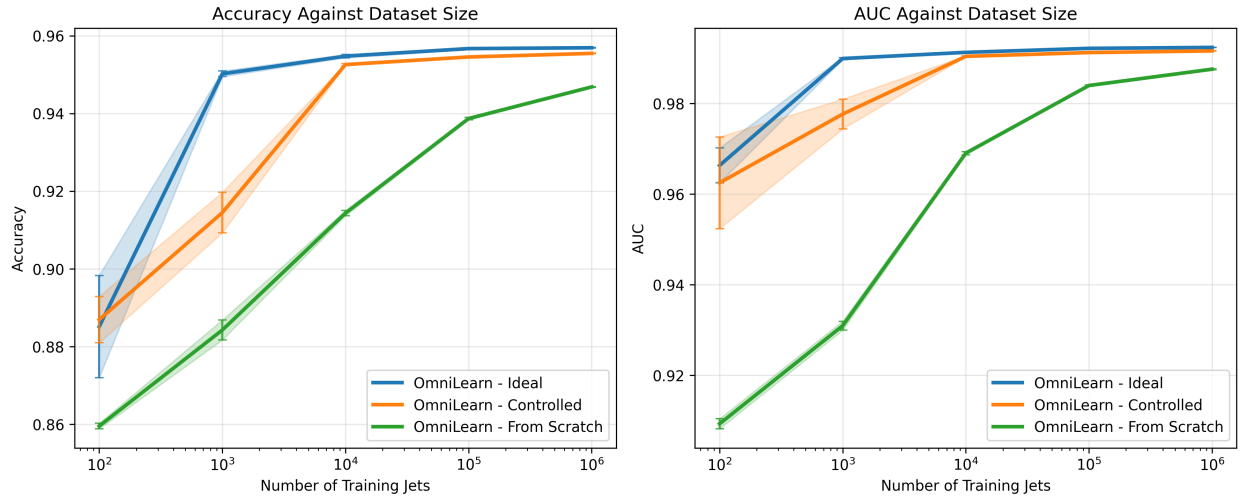


Figure 3.1: OmniLearn shows a top-tagging advantage in transfer learning over from-scratch training {CC, EE}.

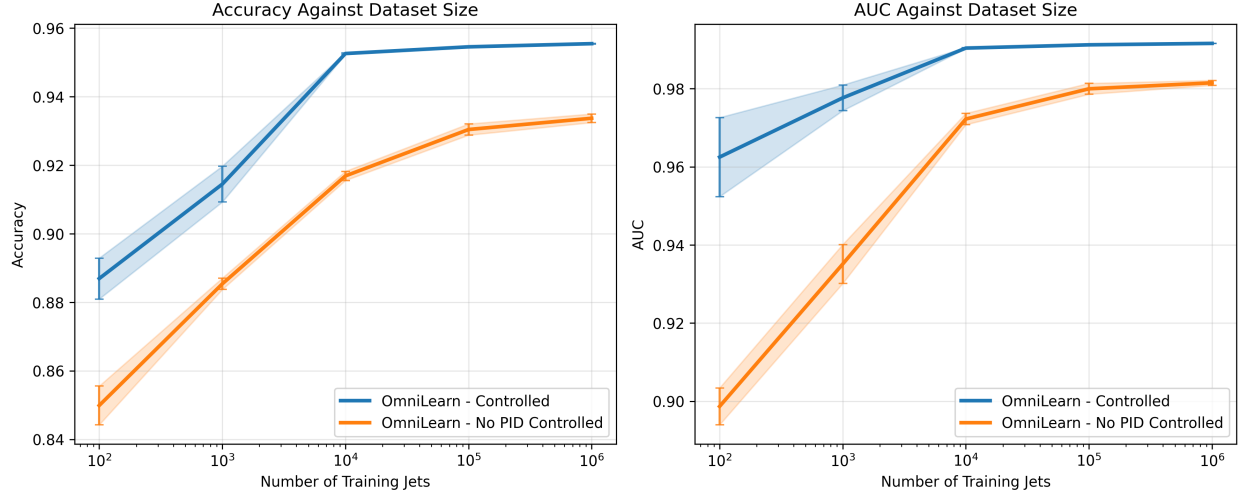


Figure 3.2: Removing additional particle inputs reduces OmniLearn’s top-tagging performance $\{\text{EE}\}$.

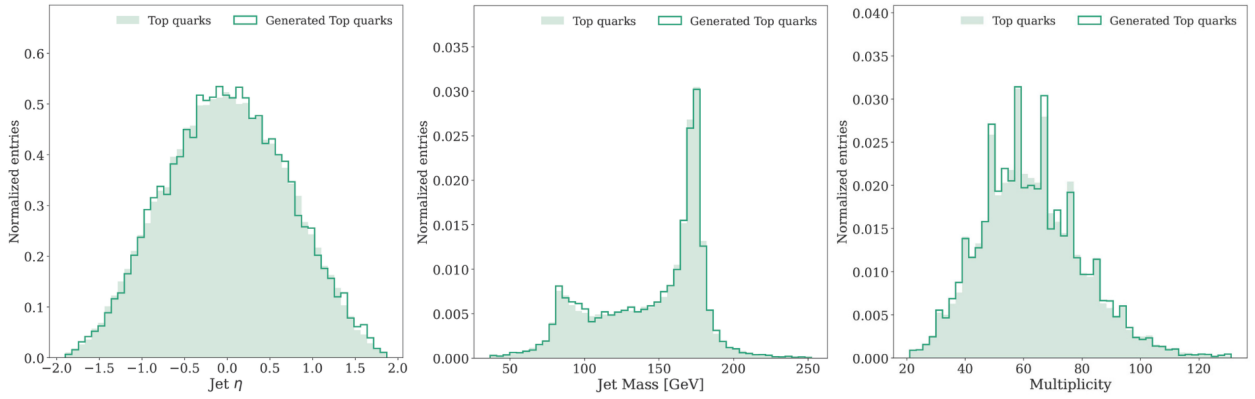


Figure 3.3: OmniLearn’s generative performance $\{\text{EE}\}$.

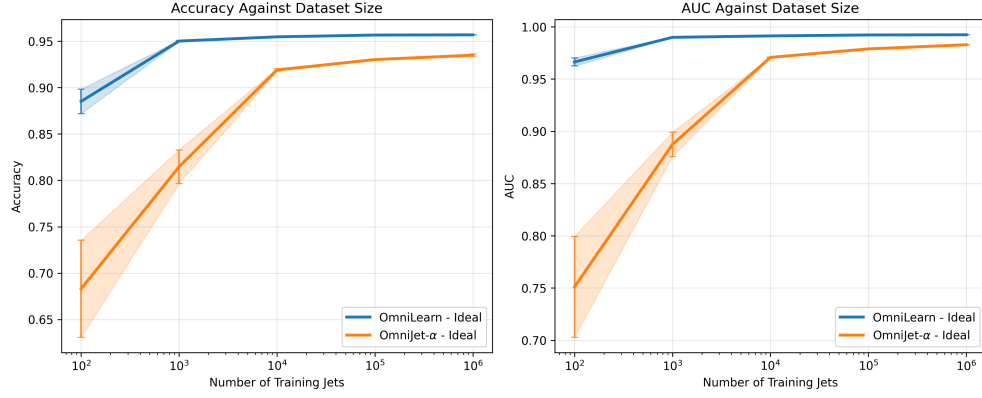


Figure 4.1: Top-tagging ACC and AUC for OmniJet- α and OmniLearn using the ideal backbone {BB, EE}.

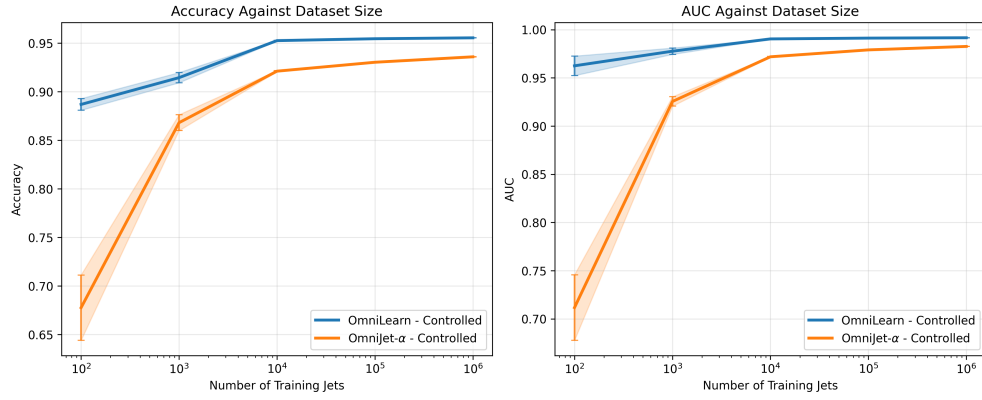


Figure 4.2: Top-tagging ACC and AUC for OmniJet- α and OmniLearn using the controlled backbone {CC, EE}.

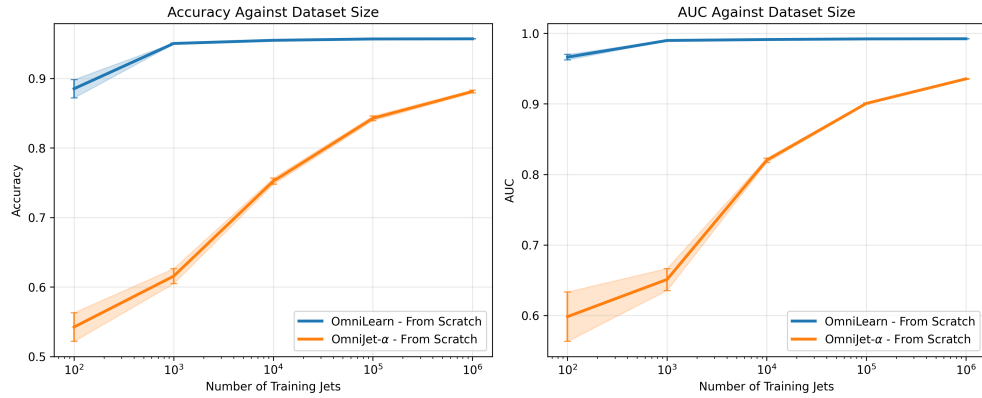


Figure 4.3: Top-tagging ACC and AUC for OmniJet- α and OmniLearn trained from scratch {DD, EE}.

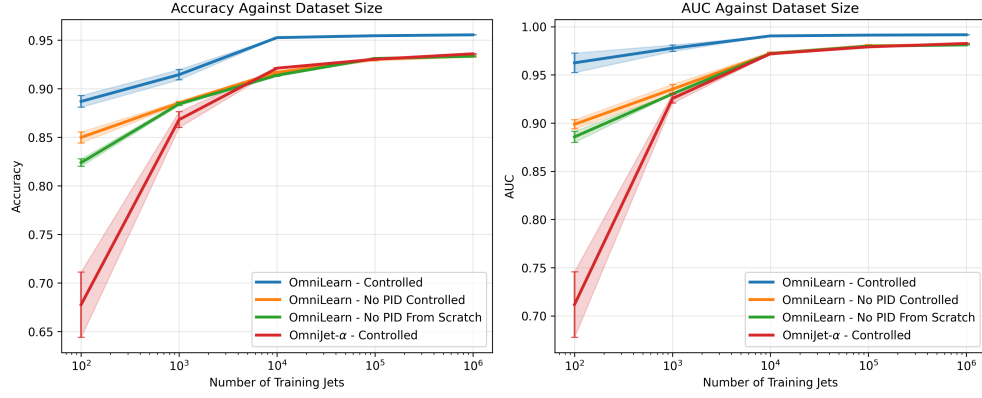


Figure 4.4: Top-tagging ACC and AUC against dataset size with and without OmniLearn’s additional inputs {DD, EE}.

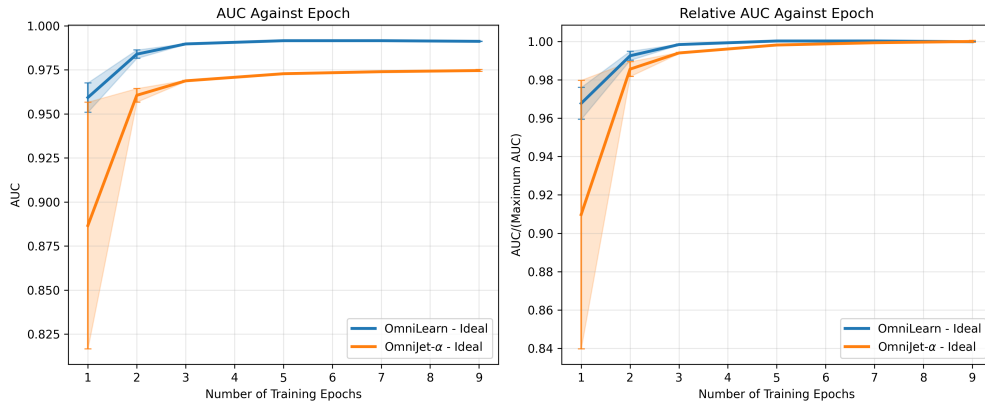


Figure 4.5: Top-tagging AUC against epochs for OmniJet- α and OmniLearn {EE}.

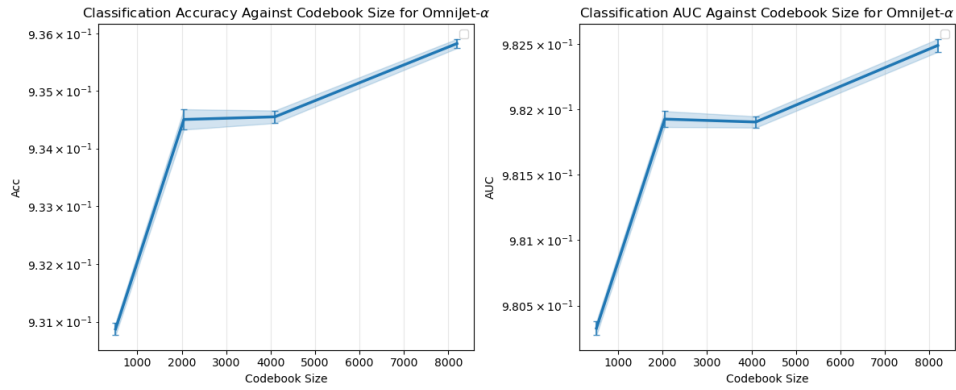


Figure 4.6: Top-tagging ACC and AUC of OmniJet- α against codebook size {AA}.

Jet Type	OmniLearn	OmniJet- α
$t \rightarrow bqq'$	0.95547 ± 0.00009	0.9362 ± 0.0001
$H \rightarrow b\bar{b}$	0.94017 ± 0.00006	0.8920 ± 0.0003
$t \rightarrow bl\nu$	0.99759 ± 0.00004	0.96986 ± 0.00008
$W \rightarrow qq'$	0.9091 ± 0.0001	0.88672 ± 0.00007
$Z \rightarrow qq'$	0.8998 ± 0.0001	0.87469 ± 0.00009

Table 1: Peak ACC of OmniLearn and OmniJet- α across jet types {AA, EE}.

Jet Type	OmniLearn	OmniJet- α
$t \rightarrow bqq'$	0.991594 ± 0.000009	0.98265 ± 0.00006
$H \rightarrow b\bar{b}$	0.985926 ± 0.000010	0.96014 ± 0.00006
$t \rightarrow bl\nu$	0.999956 ± 0.000001	0.99418 ± 0.00004
$W \rightarrow qq'$	0.97000 ± 0.00002	0.94712 ± 0.00006
$Z \rightarrow qq'$	0.96521 ± 0.00002	0.94712 ± 0.00007

Table 2: Peak AUC of OmniLearn and OmniJet- α across jet types {AA, EE}.

epochs of fine-tuning we use in our other tests allows for near-maximum performance.

Figure 4.6 shows a visible correlation between VQ-VAE codebook size and classifier performance, with a dominant step between 512 and 2048 tokens. This impact is only marginal, however, as changes in accuracy and AUC are in the order of 10^{-3} .

4 Discussion {AA, EE}

Our results clearly demonstrate that both models fulfil the foundation model directive; in each case, pretraining on a larger dataset and then fine-tuning with 10^3 jets produces the same classification performance as training from-scratch on 10^6 jets. This is valuable in HEP where labelled jet data is scarce for many specific jet type and detector combinations, and where large computational demands mean scalability

is critical [24, 27]. Both models also derive improved peak performance from transfer learning at the largest dataset sizes, after performance had clearly plateaued. Such downstream benefits may be attributed to pretraining enabling more complete learning of jet structure. Both models can also generate jets, demonstrating the cross-task capability that underlines the value of foundational models.

Our results robustly demonstrate that OmniLearn outperforms OmniJet on classification transfer learning across jet types, dataset sizes, and backbone setups. OmniLearn’s inclusion of additional input information explicitly discarded by OmniJet- α almost fully explains the peak performance discrepancy at dataset sizes over 10^3 jets. In the small-data regime, however, differences persist after the exclusion of extra input information, even when training from-scratch, indicating that pretraining advantage accounts

for very little of this difference. Tokenization was originally hypothesised as a representation bottleneck that limits how well the OmniJet- α backbone can learn jet structure, which could potentially explain this result. However, while our results show a measurable increase in classification accuracy with codebook size, the gain is much too small to explain the observed performance gap. It is therefore likely that qualitative architecture differences play more of a role, in particular, the non-sequential PET model architecture enabling relationships between all particles to be learned more rapidly [15].

Alignment of the model’s data representation with the natural structure of jets is highlighted in the OmniLearn paper as a key advantage. While this is consistent with our results, and other findings that using physical principles to guide models improves performance [28, 29], our results also suggest that the OmniJet- α architecture may be capable of identical peak performance if additional input is included. Sufficient training data may thus enable models to “learn” jet physics from scratch, overcoming a lack of prior guidance.

The importance of particle input information for performance has further implications for the ideal model architecture. Although OmniJet- α may achieve the same performance with more inputs, adding input dimensions exponentially increases the size of the input feature space tokenisation must cover. This would result in either a much larger model, strained computational resources, or compression that becomes increasingly acute. Furthermore, for models like OmniJet- α all neural networks must be retrained whenever the tokenisation regime changes, reducing its transferability to different input datasets from real detectors and undermining the vision of a multi-purpose backbone. The

OmniLearn pipeline, by comparison, adapts to any set of input features during fine-tuning.

While we have produced a comprehensive account of classifier performance, future investigations could delve deeper into generative performance, which may have different key determinants. Tokenization loss, for example, could more severely impact particle-level generation, even if losses average-out more over an entire jet. Looking beyond these models, building new tokenized models which consider more particle features would be valuable. If the performance gap to OmniLearn can be closed successfully it would confirm that effective future models must maximise particle-level input spaces.

5 Conclusion {AA, BB, CC, EE}

We have demonstrated that both OmniJet- α and OmniLearn represent viable foundation model architectures, capable of generation and tagging with performance gains from transfer learning.

OmniJet- α adopts the approach of LLMs, but while sentences are discrete and sequential, lending themselves to tokenisation and autoregressive models,[30] jets are unordered with continuous features, seemingly demanding different representations [29]. However, the observed advantage of aligning model representation with jet structure was negligible on datasets over just 10^4 jets with the same particle-level inputs. It therefore seems likely that LLM inspired models will be highly effective for all but the most data-constrained tasks. This highlights that the particle input space is a more important consideration for maximising the performance of future models with any architecture.

Nonetheless, we suggest that OmniLearn’s ar-

chitecture remains a more promising candidate for the future of jet foundation models, due to its greater demonstrated flexibility across tasks and input features. OmniLearn has indeed already inspired a flagship successor, OmniLearned, which builds on these principles and outperforms most of its specialised deep learning counterparts [23].

References

- [1] CMS Collaboration. “Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC”. In: *Physics Letters B* 716.1 (2012), pp. 30–61. DOI: [10.1016/j.physletb.2012.08.021](https://doi.org/10.1016/j.physletb.2012.08.021). URL: <https://www.sciencedirect.com/science/article/pii/S0370269312008581>.
- [2] CERN. *Supersymmetry*. 2019. URL: <https://home.cern/science/physics/supersymmetry>.
- [3] CMS Collaboration. “Search for dark QCD with emerging jets in proton-proton collisions at $\sqrt{s} = 13$ TeV”. In: *Journal of High Energy Physics* 07 (2024), p. 142. DOI: [10.1007/JHEP07\(2024\)142](https://doi.org/10.1007/JHEP07(2024)142). arXiv: [2403.01556](https://arxiv.org/abs/2403.01556) [hep-ex]. URL: <https://arxiv.org/abs/2403.01556>.
- [4] CMS Collaboration. *Jet cones with top flavour*. CMS Experiment, Accessed 15 Mar. 2026. 2022. URL: <https://cms.cern/news/jet-cones-top-flavour>.
- [5] Jeff Greensite and Kazue Matsuyama. “On the distinction between color confinement, and confinement”. In: *arXiv preprint* (2018). arXiv: [1811.01512](https://arxiv.org/abs/1811.01512) [hep-lat]. URL: <https://arxiv.org/abs/1811.01512>.
- [6] ATLAS Collaboration. *Machine learning is revolutionising our understanding of particle “jets”*. Physics briefing, ATLAS Experiment at CERN. 2023. URL: <https://atlas.cern/Updates/Briefing/Boosting-Jets-AI>.
- [7] Riccardo Di Sipio et al. “DijetGAN: A Generative-Adversarial Network Approach for the Simulation of QCD Dijet Events at the LHC”. In: *Journal of High Energy Physics* 2019.08 (2019), p. 110. DOI: [10.1007/JHEP08\(2019\)110](https://doi.org/10.1007/JHEP08(2019)110). arXiv: [1903.02433](https://arxiv.org/abs/1903.02433) [hep-ex]. URL: <https://arxiv.org/abs/1903.02433>.
- [8] Rajan Gupta, Sanju Tiwari, and Poonam Chaudhary. “Foundation Models”. In: *Generative AI: Techniques, Models and Applications*. Cham: Springer Nature Switzerland, 2025, pp. 65–79. ISBN: 978-3-031-82062-5. DOI: [10.1007/978-3-031-82062-5_4](https://doi.org/10.1007/978-3-031-82062-5_4). URL: https://doi.org/10.1007/978-3-031-82062-5_4.
- [9] Johannes Schneider, Christian Meske, and Pauline Kuss. “Foundation Models”. eng. In: *Business Information Systems Engineering* 66.2 (2024), pp. 221–231. ISSN: 1867-0202. DOI: [10.1007/s12599-024-00851-0](https://doi.org/10.1007/s12599-024-00851-0). URL: <https://hdl.handle.net/10419/315722>.
- [10] J. Birk, A. Hallin, G. Kasieczka, et al. “Enhancing next token prediction based pre-training for jet foundation models”. In: *arXiv preprint* (2025). DOI: [10.48550/arXiv.2512.04149](https://doi.org/10.48550/arXiv.2512.04149). arXiv: [2512.04149](https://arxiv.org/abs/2512.04149) [hep-ph]. URL: <https://arxiv.org/abs/2512.04149>.

- [11] M. Radova, W.G. Stark, C.S. Allen, et al. “Fine-tuning foundation models of materials interatomic potentials with frozen transfer learning”. In: *npj Computational Materials* 11 (2025), p. 237. DOI: [10.1038/s41524-025-01727-x](https://doi.org/10.1038/s41524-025-01727-x). URL: <https://doi.org/10.1038/s41524-025-01727-x>.
- [12] Jesse Thaler and Ken Van Tilburg. “Maximizing boosted top identification by minimizing N -subjettiness”. In: *Journal of High Energy Physics* 2012.2 (2012), p. 093. DOI: [10.1007/JHEP02\(2012\)093](https://link.springer.com/article/10.1007/JHEP02(2012)093). URL: [https://link.springer.com/article/10.1007/JHEP02\(2012\)093](https://link.springer.com/article/10.1007/JHEP02(2012)093).
- [13] Huilin Qu and Loukas Gouskos. “Jet tagging via particle clouds”. In: *Physical Review D* 101 (2020), p. 056019. DOI: [10.1103/PhysRevD.101.056019](https://doi.org/10.1103/PhysRevD.101.056019). URL: <http://dx.doi.org/10.1103/PhysRevD.101.056019>.
- [14] Spandan Mondal and Luca Mastrolorenzo. “Machine learning in high energy physics: a review of heavy-flavor jet tagging at the LHC”. In: *European Physical Journal Special Topics* 233 (2024), pp. 2657–2686. DOI: [10.1140/epjs/s11734-024-01234-y](https://link.springer.com/article/10.1140/epjs/s11734-024-01234-y). URL: <https://link.springer.com/article/10.1140/epjs/s11734-024-01234-y>.
- [15] V. Mikuni and B. Nachman. “OmniLearn: A Method to simultaneously facilitate all jet physics tasks”. In: *Physical Review D* 111.5 (2025), p. 054015. DOI: [10.1103/physrevd.111.054015](https://escholarship.org/uc/item/6gd219b8). URL: <https://escholarship.org/uc/item/6gd219b8>.
- [16] Joschka Birk, Anna Hallin, and Gregor Kasieczka. “OmniJet-: the first cross-task foundation model for particle physics”. In: *Machine Learning: Science and Technology* 5.3 (Aug. 2024), p. 035031. ISSN: 2632-2153. DOI: [10.1088/2632-2153/ad66ad](https://doi.org/10.1088/2632-2153/ad66ad). URL: <http://dx.doi.org/10.1088/2632-2153/ad66ad>.
- [17] Jonathan Ho, Ajay Jain, and Pieter Abbeel. “Denoising Diffusion Probabilistic Models”. In: *arXiv preprint* (2020). DOI: [10.48550/arXiv.2006.11239](https://arxiv.org/abs/2006.11239). arXiv: [2006.11239](https://arxiv.org/abs/2006.11239) [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [18] Oona Rainio, Jarmo Teuvo, and Riku Klén. “Evaluation metrics and statistical tests for machine learning”. In: *Scientific Reports* 14.1 (2024), p. 6086. DOI: [10.1038/s41598-024-56706-x](https://doi.org/10.1038/s41598-024-56706-x). URL: <https://doi.org/10.1038/s41598-024-56706-x>.
- [19] Jing Li. “Area under the ROC Curve has the most consistent evaluation for binary classification”. In: *PLOS One* 19.12 (2024), e0316019. DOI: [10.1371/journal.pone.0316019](https://pmc.ncbi.nlm.nih.gov/articles/PMC11666033/). URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC11666033/>.
- [20] Larry Wasserman. *Optimal Transport and Wasserstein Distance*. Lecture notes, Carnegie Mellon University. Available at <https://www.stat.cmu.edu/~larry/=sml/Opt.pdf>. 2019.
- [21] Oz Amram et al. “Aspen Open Jets: Unlocking LHC Data for Foundation Models in Particle Physics”. In: *arXiv preprint* (2024). DOI: [10.48550/arXiv.2412.10504](https://arxiv.org/abs/2412.10504). arXiv: [2412.10504](https://arxiv.org/abs/2412.10504) [hep-ph]. URL: <https://arxiv.org/abs/2412.10504>.
- [22] Huilin Qu. *Advanced jet tagging with ML*. CERN Physics Department - LPCC Seminar. Presented at CERN, Geneva,

- Switzerland. Sept. 2023. URL: https://indico.cern.ch/event/1327524/attachments/2721386/4727952/jet_tagging_20230925_H_Qu.pdf.
- [23] Wahid Bhimji et al. “OmniLearned: A Foundation Model Framework for All Tasks Involving Jet Physics”. In: (2025). DOI: [10.48550/arXiv.2510.24066](https://arxiv.org/abs/2510.24066). arXiv: [2510.24066](https://arxiv.org/abs/2510.24066) [hep-ph]. URL: <https://arxiv.org/abs/2510.24066>.
- [24] Huilin Qu, Congqiao Li, Sitian Qian, et al. “Particle Transformer for Jet Tagging”. In: *arXiv preprint* (2022). DOI: [10.48550/arXiv.2202.03772](https://arxiv.org/abs/2202.03772). arXiv: [2202.03772](https://arxiv.org/abs/2202.03772) [hep-ph]. URL: <https://arxiv.org/abs/2202.03772>.
- [25] Joep Geuskens et al. “Fundamental limit of jet tagging”. In: *Physical Review D* 112.9 (2025), p. L091901. DOI: [10.1103/mjj1-w2b1](https://journals.aps.org/prd/abstract/10.1103/mjj1-w2b1). URL: <https://journals.aps.org/prd/abstract/10.1103/mjj1-w2b1>.
- [26] Jared Kaplan et al. “Scaling Laws for Neural Language Models”. In: *arXiv preprint* (2020). DOI: [10.48550/arXiv.2001.08361](https://arxiv.org/abs/2001.08361). arXiv: [2001.08361](https://arxiv.org/abs/2001.08361) [cs.LG]. URL: <https://arxiv.org/abs/2001.08361>.
- [27] Alexander Radovic et al. “Machine learning at the energy and intensity frontiers of particle physics”. In: *Nature* 560.7716 (2018), pp. 41–48. DOI: [10.1038/s41586-018-0361-2](https://www.osti.gov/biblio/1469751). URL: <https://www.osti.gov/biblio/1469751>.
- [28] Patrick T. Komiske, Eric M. Metodiev, and Jesse Thaler. “Energy Flow Networks: Deep Sets for Particle Jets”. In: *Journal of High Energy Physics* 01 (2019), p. 121. DOI: [10.1007/JHEP01\(2019\)121](https://arxiv.org/abs/1810.05165). arXiv: [1810.05165](https://arxiv.org/abs/1810.05165) [hep-ph]. URL: <https://arxiv.org/abs/1810.05165>.
- [29] Huilin Qu and Loukas Gouskos. “ParticleNet: Jet Tagging via Particle Clouds”. In: *Physical Review D* 101.5 (2020), p. 056019. DOI: [10.1103/PhysRevD.101.056019](https://arxiv.org/abs/1902.08570). arXiv: [1902.08570](https://arxiv.org/abs/1902.08570) [hep-ph]. URL: <https://arxiv.org/abs/1902.08570>.
- [30] Alec Radford et al. *Language models are unsupervised multitask learners*. 2019. URL: <https://www.semanticscholar.org/paper/Language-Models-are-Unsupervised-Multitask-Learners-Radford-Wu/9405cc0d6169988371b2755e573cc28650d14dfe> (visited on 01/06/2023).
- [31] B. Orzari, N. Chernyavskaya, R. Cobe, et al. “LHC hadronic jet generation using convolutional variational autoencoders with normalizing flows”. In: *Machine Learning: Science and Technology* 4.4 (2023), p. 045023. DOI: [10.1088/2632-2153/ad04ea](https://dx.doi.org/10.1088/2632-2153/ad04ea). URL: [http://dx.doi.org/10.1088/2632-2153/ad04ea](https://dx.doi.org/10.1088/2632-2153/ad04ea).

Appendix A: Software Development

The pages below contain the code developed to produce our results. Most are modified scripts adapting OmniLearn to use JetClass data for classification testing {EE}.

Preprocessing OmniLearn Data:

This script preprocesses JetClass data to make it suitable for OmniLearn’s training and evaluation scripts. It has been modified from the original to replace multi-class labels with binary labels, as OmniLearn’s evaluation only supports binary classification. The binary labels are assigned as “0” for quark/gluon jets and “1” for any other type. It also can cut off the additional input information OmniLearn uses natively to create the “No PID” dataset.

```
import os
import numpy as np
import h5py
from sklearn.model_selection import KFold
from dataloader import read_file
import gc
from optparse import OptionParser

def process_and_save(folder_path, n_splits, name):
    all_files = [os.path.join(folder_path, f) for f in
os.listdir(folder_path) if os.path.isfile(os.path.join(folder_path, f))]

    output_dir = os.path.join(folder_path, name)
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    kf = KFold(n_splits=n_splits)

    for i, (_, split_indices) in enumerate(kf.split(all_files)):
        print("Running Fold {}".format(i))
        concat_X = []
        concat_y = []
        concat_j = []
        for idx in split_indices:
            file_path = all_files[idx]
            X, j, y = read_file(file_path)
```

```

        ### This part swaps out the multi-class labels for binary
labels
        pid_value = 0 if "ZJetsToNuNu" in file_path else 1
        y = np.full((y.shape[0], 1), pid_value, dtype=np.int8)

        concat_X.append(X)
        concat_y.append(y)
        concat_j.append(j)

        final_X = np.concatenate(concat_X, axis=0)
        final_X = final_X[:, :, :3]  ### Here we cut the particle-level
data to contain only the same inputs as OmniJet
        final_y = np.concatenate(concat_y, axis=0)
        final_j = np.concatenate(concat_j, axis=0)

        # Save to h5py file
        with h5py.File('{}/{}/JetClass_{}.h5'.format(folder_path, name,
i), 'w') as h5f:
            h5f.create_dataset('data', data=final_X)
            h5f.create_dataset('jet', data=final_j)
            h5f.create_dataset('pid', data=final_y)
        del final_X, final_y, concat_X, concat_y
        gc.collect()

if __name__ == '__main__':
    parser = OptionParser(usage="%prog [opt] inputFiles")
    parser.add_option("--sample", type="string", default='train',
help="Input file name")
    parser.add_option("--folder", type="string",
default='/pscratch/sd/v/vmikuni/PET/JetClass', help="Folder containing
downloaded files")

```

```

(flags, args) = parser.parse_args()

if flags.sample == 'train':
    process_and_save(os.path.join(flags.folder, 'train_100M'),
n_splits=20, name='train')
    if flags.sample == 'test':
        process_and_save(os.path.join(flags.folder, 'test_20M'),
n_splits=40, name='test')
    if flags.sample == 'val':
        process_and_save(os.path.join(flags.folder, 'val_5M'),
n_splits=10, name='val')

```

Loading Data for OmniLearn

This next script is a dataloader adapted from multiple examples in the OmniLearn repository. It passes multiple input files to the training script, as suited for JetClass data, but concatenates testing files together for evaluation. It is also where modifications can be made to the dataset size used by OmniLearn, and where we can identify to the training and classification scripts that binary class labels have been applied.

```

class JetTestTrainLoader(DataLoader):
    def __init__(self, path, batch_size=512, rank=0, size=1,
chunk_size=5000):
        super().__init__(path, batch_size, rank, size)

        self.chunk_size = chunk_size
        all_files = [os.path.join(self.path, f) for f in os.listdir(path)
if os.path.isfile(os.path.join(path, f))]
        self.files = np.array_split(all_files, self.size)[self.rank]

        self.get_stats(all_files)

    ### This is the key place to adjust how much data is being

```

```

### considered
def get_stats(self, file_list):
    #Will assume each file is 100k long
    self.nevts = len(file_list)*100000//5
    self.num_part = h5.File(file_list[0], 'r')['data'].shape[1]
    self.num_feat = h5.File(file_list[0], 'r')['data'].shape[2]
    self.num_jet = 4 #hardcoded for convenience
    self.num_classes = 2 ### This was hard-coded to prevent bugs
    self.steps_per_epoch = 1900
    self.num_pad = 0

def single_file_generator(self, file_path):

    with h5.File(file_path, "r") as file:

        data_size = file["data"].shape[0]

        for start in range(0, data_size, self.chunk_size):
            end = min(start + self.chunk_size, data_size)

            data_chunk = file["data"][start:end].astype(np.float32)
            jet_chunk = file["jet"][start:end]
            label_chunk = file["pid"][start:end]

            mask_particle = jet_chunk[:, -1] > 1

            data_chunk = data_chunk[mask_particle]
            jet_chunk = jet_chunk[mask_particle]
            label_chunk = label_chunk[mask_particle]

            if data_chunk.shape[0] == 0:
                continue

            mask_chunk = data_chunk[:, :, 2] != 0

```

```

        data_chunk = self.preprocess(data_chunk,
mask_chunk).astype(np.float32)
        data_chunk = self.pad(data_chunk, num_pad=self.num_pad)

        jet_chunk =
self.preprocess_jet(jet_chunk).astype(np.float32)

        label_chunk = label_chunk.astype(np.int32).reshape(-1)
        label_chunk = np.eye(2)[label_chunk]

        points_chunk = data_chunk[:, :, :2]

        for j in range(data_chunk.shape[0]):
            yield (
                {
                    "input_features": data_chunk[j],
                    "input_points": points_chunk[j],
                    "input_mask":
mask_chunk[j].astype(np.float32),
                    "input_jet": jet_chunk[j],
                },
                label_chunk[j],
            )

### This produces multiple files for fine-tuning
def interleaved_file_generator(self):

    random.shuffle(self.files)

    generators = [
        self.single_file_generator(fp)
        for fp in self.files
    ]

    round_robin = itertools.cycle(generators)

```

```

while True:
    try:
        gen = next(round_robin)
        yield next(gen)
    except StopIteration:
        break

def make_tfdata(self):

    dataset = tf.data.Dataset.from_generator(
        self.interleaved_file_generator,
        output_signature=(
            {
                "input_features": tf.TensorSpec(
                    shape=(self.num_part, self.num_feat),
                    dtype=tf.float32,
                ),
                "input_points": tf.TensorSpec(
                    shape=(self.num_part, 2),
                    dtype=tf.float32,
                ),
                "input_mask": tf.TensorSpec(
                    shape=(self.num_part, ),
                    dtype=tf.float32,
                ),
                "input_jet": tf.TensorSpec(
                    shape=(self.num_jet, ),
                    dtype=tf.float32,
                ),
            },
            tf.TensorSpec(
                shape=(self.num_classes, ),
                dtype=tf.float32,
            ),

```

```

        ),
    )

    return (
        dataset
        .shuffle(self.batch_size * 50)
        .repeat()
        .batch(self.batch_size)
        .prefetch(tf.data.AUTOTUNE)
    )

### This function produces a concatenated file for evaluation
def make_eval_data(self):
    X_list = []
    y_list = []

    for file_path in self.files:
        with h5.File(file_path, "r") as file:
            n_rows = min(1000, file["data"].shape[0])
            data = file["data"][:n_rows]
            jet = file["jet"][:n_rows]
            label = file["pid"][:n_rows]

            # Apply same preprocessing as training
            mask_particle = jet[:, -1] > 1
            data = data[mask_particle]
            jet = jet[mask_particle]
            label = label[mask_particle]

            mask = data[:, :, 2] != 0

            data = self.preprocess(data, mask)
            data = self.pad(data, num_pad=self.num_pad)
            jet = self.preprocess_jet(jet)

```

```

        label = label.astype(np.int32).reshape(-1)
        label = np.eye(self.num_classes)[label]

        points = data[:, :, :2]

        X_list.append({
            "input_features": data,
            "input_points": points,
            "input_mask": mask.astype(np.float32),
            "input_jet": jet,
            "input_time": np.zeros((data.shape[0], 1),
dtype=np.float32),
        })

        y_list.append(label)

    ### Concatenation
    X = {
        key: np.concatenate([x[key] for x in X_list], axis=0)
        for key in X_list[0]
    }
    y = np.concatenate(y_list, axis=0)

    return X, y

```

Evaluating OmniLearn

This modified evaluation script is set up to load a model from a path it is given when the function is called. This replaced its previous automatic loading that could only cope with one model checkpoint being saved at a time. It was similarly modified to write results out to a file rather than simply printing them. This allows multiple checkpoints to be evaluated simultaneously and was essential for high-volume testing.

```

import os
import numpy as np

```



```

import tensorflow as tf
from tensorflow import keras
from sklearn import metrics
import horovod.tensorflow.keras as hvd
import argparse
import sys
import gc
from pathlib import Path

import h5py
from PET import PET
import utils
from omnifold import Classifier

def parse_arguments():
    parser = argparse.ArgumentParser(description="Evaluate performance
metrics for trained models on various datasets.")
    parser.add_argument("--dataset", type=str, default="top", help="Folder
containing input files")
    parser.add_argument("--folder", type=str,
default="/pscratch/sd/v/vmikuni/PET/", help="Folder containing input
files")
    parser.add_argument("--batch", type=int, default=5000, help="Batch
size")
    parser.add_argument("--load", action='store_true', help="Load
pre-evaluated npy files")
    parser.add_argument("--mode", type=str, default="classifier",
help="Loss type to train the model")
    parser.add_argument("--fine_tune", action='store_true', help="Fine
tune a model")
    parser.add_argument("--nid", type=int, default=0, help="Training ID
for multiple trainings")
    parser.add_argument("--local", action='store_true', help="Use local
embedding")

```

```

    parser.add_argument("--num_layers", type=int, default=8, help="Number
of transformer layers")
    parser.add_argument("--drop_probability", type=float, default=0.0,
help="Stochastic Depth drop probability")
    parser.add_argument("--simple", action='store_true', help="Use
simplified head model")
    parser.add_argument("--talking_head", action='store_true', help="Use
talking head attention")
    parser.add_argument("--layer_scale", action='store_true', help="Use
layer scale in the residual connections")
    ### Model path introduced to allow multiple saved checkpoints
    parser.add_argument("--model_path", type=str, help="Path to the
trained model checkpoint (.ckpt or .pt)")
    args = parser.parse_args()
    return args

def print_metrics(y_pred, y, thresholds, model_path, multi_label=False):
    if multi_label:
        print("AUC: {}".format(metrics.roc_auc_score(y,
y_pred, average='macro', multi_class='ovo'))))

        one_hot_predictions = np.zeros_like(y_pred)
        one_hot_predictions[np.arange(len(y_pred)),
y_pred.argmax(axis=-1)] = 1

        print('Acc:
{}'.format(metrics.accuracy_score(y, one_hot_predictions)))

    bkg_idx = 0

    for idx in range(np.shape(y)[-1]):
        if idx == bkg_idx: continue
        mask = (y[:,idx]==1) | (y[:,bkg_idx]==1) #only keep signal+bkg

```

```

        pred_sb = y_pred[mask,idx]/(y_pred[mask,idx] +
y_pred[mask,bkg_idx])
        fpr, tpr, _ = metrics.roc_curve(y[mask,idx], pred_sb)

        for threshold in thresholds:
            bineff = np.argmax(tpr>threshold)
            print('Class {} effS at {} 1.0/effB =
{}'.format(idx,tpr[bineff],1.0/fpr[bineff]))

    ### This modification makes sure metrics are written to a file
    rather than just printed
    output_file = Path(model_path).with_suffix(".txt")

    with open(output_file, "w") as f:

        auc = metrics.roc_auc_score(
            y, y_pred, average='macro', multi_class='ovo'
        )
        f.write("AUC: {}\n".format(auc))

        one_hot_predictions = np.zeros_like(y_pred)
        one_hot_predictions[np.arange(len(y_pred)),
y_pred.argmax(axis=-1)] = 1

        acc = metrics.accuracy_score(y, one_hot_predictions)
        f.write("Acc: {}\n".format(acc))

        bkg_idx = 0

        for idx in range(np.shape(y)[-1]):
            if idx == bkg_idx:
                continue

            mask = (y[:, idx] == 1) | (y[:, bkg_idx] == 1)

```

```

pred_sb = y_pred[mask, idx] / (
    y_pred[mask, idx] + y_pred[mask, bkg_idx]
)

fpr, tpr, _ = metrics.roc_curve(y[mask, idx], pred_sb)

for threshold in thresholds:
    bineff = np.argmax(tpr > threshold)
    if bineff < len(fpr) and fpr[bineff] > 0:
        f.write(
            "Class {} effS at {} 1.0/effB = {}\n".format(
                idx, tpr[bineff], 1.0 / fpr[bineff]
            )
        )
    else:
        f.write(
            "Class {} effS at {} 1.0/effB =
undefined\n".format(
                idx, threshold
            )
        )

print(f"Multi-label metrics written to {output_file}")

else:
    print("AUC: {}".format(metrics.roc_auc_score(y, y_pred)))
    print('Acc: {}'.format(metrics.accuracy_score(y, y_pred>0.5)))
    Auc = metrics.roc_auc_score(y, y_pred)
    Acc = metrics.accuracy_score(y, y_pred>0.5)
    fpr, tpr, _ = metrics.roc_curve(y, y_pred)

    for threshold in thresholds:
        bineff = np.argmax(tpr>threshold)
        print('effS at {} 1.0/effB =
{}'.format(tpr[bineff], 1.0/fpr[bineff]))

```

```

output_file = Path(model_path).with_suffix(".txt")

with open(output_file, "w") as f:
    auc = metrics.roc_auc_score(y, y_pred)
    acc = metrics.accuracy_score(y, y_pred > 0.5)
    fpr, tpr, _ = metrics.roc_curve(y, y_pred)

    f.write("AUC: {}\n".format(auc))
    f.write("Acc: {}\n".format(acc))

    for threshold in thresholds:
        bineff = np.argmax(tpr > threshold)
        if bineff < len(fpr) and fpr[bineff] > 0:
            f.write("effS at {} 1.0/effB = {}\n".format(
                tpr[bineff], 1.0 / fpr[bineff]
            ))
        else:
            f.write("effS at {} 1.0/effB =
undefined\n".format(threshold))

print(f"Metrics written to {output_file}")

def get_data_info(flags):
    multi_label = True
    if flags.dataset == 'top':
        test = utils.TopDataLoader(os.path.join(flags.folder, 'TOP',
'test_ttbar.h5'),
                                flags.batch, rank = hvd.rank(), size =
hvd.size())
        threshold = [0.3, 0.5]
        folder_name = 'TOP'

```

```

        if flags.dataset == 'opt':
            test = utils.TopDataLoader(os.path.join(flags.folder, 'Opt',
'test_ttbar.h5'),
                                     flags.batch, rank = hvd.rank(), size =
hvd.size())
            threshold = [0.3, 0.5]
            folder_name = 'Opt'

        elif flags.dataset == 'qg':
            test = utils.QGDataLoader(os.path.join(flags.folder, 'QG',
'test_qg.h5'),
                                     flags.batch, rank = hvd.rank(), size =
hvd.size())
            threshold = [0.3, 0.5]
            folder_name = 'QG'

        elif flags.dataset == 'atlas':
            test = utils.AtlasDataLoader(os.path.join(flags.folder, 'ATLASTOP',
'test_atlas.h5'),
                                     flags.batch, rank = hvd.rank(), size =
hvd.size())
            threshold = [0.5, 0.8]
            folder_name = 'ATLASTOP'
            multi_label = False
        elif flags.dataset == 'atlas_small':
            test = utils.AtlasDataLoader(os.path.join(flags.folder, 'ATLASTOP',
'test_atlas.h5'),
                                     flags.batch, rank = hvd.rank(), size =
hvd.size())
            threshold = [0.5, 0.8]
            folder_name = 'ATLASTOP'
            multi_label = False
        elif flags.dataset == 'h1':

```

```

        test = utils.H1DataLoader(os.path.join(flags.folder, 'H1',
'test.h5'),
                                flags.batch, rank = hvd.rank(), size =
hvd.size())
        threshold = [0.5, 0.1]
        folder_name = 'H1'

    elif flags.dataset == 'cms':
        test = utils.CMSQGDataLoader(os.path.join(flags.folder, 'CMSQG',
'test_qgcms_pid.h5'),
                                flags.batch, rank = hvd.rank(), size =
hvd.size())
        threshold = [0.5, 0.8]
        folder_name = 'CMSQG'

    elif flags.dataset == 'jetclass':
        test =
utils.JetClassDataLoader(os.path.join(flags.folder, 'JetClass', 'test'),
rank = hvd.rank(), size = hvd.size(), batch_size=flags.batch)
        threshold = [0.5]
        folder_name = 'JetClass/test'

### The following statements were added for the new datasets we introduced
    elif flags.dataset == 'jettest':
        test =
utils.JetTestTrainLoader(os.path.join(flags.folder, 'JetTest', 'test'),
                                rank =
hvd.rank(), size = hvd.size(),
                                batch_size=flags.batch)
        threshold = [0.3, 0.5]
        folder_name = 'JetTest/test'

    elif flags.dataset == 'jetnopicid':

```

```

        test =
utils.JetNoPidTrainLoader(os.path.join(flags.folder, 'JetNoPID', 'test'),
                           rank =
hvd.rank(), size = hvd.size(),
                           batch_size=flags.batch)

        threshold = [0.3, 0.5]
        folder_name = 'JetNoPID/test'

    elif flags.dataset == 'jethobb':
        test =
utils.JetTestTrainLoader(os.path.join(flags.folder, 'JetHTtoBB', 'test'),
                           rank =
hvd.rank(), size = hvd.size(),
                           batch_size=flags.batch)

        threshold = [0.3, 0.5]
        folder_name = 'JetHTtoBB/test'

    elif flags.dataset == 'jettoll':
        test =
utils.JetTestTrainLoader(os.path.join(flags.folder, 'JetTtoLL', 'test'),
                           rank =
hvd.rank(), size = hvd.size(),
                           batch_size=flags.batch)

        threshold = [0.3, 0.5]
        folder_name = 'JetHTtoBB/test'

    elif flags.dataset == 'jetwtoqq':
        test =
utils.JetTestTrainLoader(os.path.join(flags.folder, 'JetWtoQQ', 'test'),
                           rank =
hvd.rank(), size = hvd.size(),
                           batch_size=flags.batch)

        threshold = [0.3, 0.5]
        folder_name = 'JetHTtoBB/test'

```



```

        elif flags.dataset == 'jetztoqq':
            test =
utils.JetTestTrainLoader(os.path.join(flags.folder, 'JetZtoQQ', 'test'),
                           rank =
hvd.rank(), size = hvd.size(),
                           batch_size=flags.batch)

            threshold = [0.3, 0.5]
            folder_name = 'JetHToBB/test'

        return test, multi_label, threshold, folder_name

def get_model_function(dataset):
    if 'atlas' in dataset:
        return Classifier, 'sigmoid'
    else:
        return PET, 'softmax'

def load_or_evaluate_model(flags, test, folder_name):
    npy_file = os.path.join(flags.folder, folder_name, 'np', '{}'.format(
        utils.get_model_name(
            flags, fine_tune=flags.fine_tune,
            add_string = '{}'.format(flags.nid) if flags.nid > 0 else
            '')).replace('.h5', '.np'))

    if flags.load:
        print("Loading saved np files")
        data = np.load(npy_file, allow_pickle=True)
        return data.item()['y'], data.item()['pred']
    else:
        model_function, activation = get_model_function(flags.dataset)
        model = model_function(num_feat=test.num_feat,
                               num_jet=test.num_jet,
                               num_classes=test.num_classes,

```

```

                                local=flags.local,
num_layers=flags.num_layers,
                                drop_probability=flags.drop_probability,
                                simple=flags.simple,
layer_scale=flags.layer_scale,
                                talking_head=flags.talking_head,
                                mode=flags.mode,
class_activation=activation)

    X, y = test.make_eval_data()
    if flags.nid>0:
        #Load alternative runs
        add_string = '_{}'.format(flags.nid)
    else:
        add_string = ''

    ### Weights are loaded from the model path passed to the
evaluation script
    model.load_weights(flags.model_path)

    y = hvd.allgather(tf.constant(y)).numpy()
    pred = hvd.allgather(tf.constant(model.predict(X,
verbose=hvd.rank() == 0)[0])).numpy()
    if hvd.rank()==0:
        if not
os.path.exists(os.path.join(flags.folder, folder_name, 'numpy')):
        os.makedirs(os.path.join(flags.folder, folder_name, 'numpy'))
        np.save(npy_file, {'y':y, 'pred':pred})

    return y, pred

def main():

```

```

utils.setup_gpus()
flags = parse_arguments()

test, multi_label, thresholds, folder_name = get_data_info(flags)

y, pred = load_or_evaluate_model(flags, test, folder_name)

print("y shape:", y.shape)
print("pred shape:", pred.shape)
print_metrics(pred, y, thresholds, flags.model_path,
multi_label=multi_label)

if __name__ == '__main__':
    main()

```

Plotting Script:

This script was written to create line plots with uncertainties shaded in. It was adapted by each group member to produce the plots throughout the report. This version produced Figure 1.1.

```

import matplotlib.pyplot as plt
import numpy as np

### X-axis values
x = np.array([100, 1000, 10000, 100000, 1000000])

### First Plot Data
y_values = [
    np.array([0.683023, 0.814579, 0.9190754, 0.9302554, 0.9350314]),
    np.array([0.6777454, 0.8681266, 0.921156, 0.9303268, 0.935931]),
    np.array([0.542351, 0.619122, 0.752104, 0.842587, 0.881177])
]

```

```

y_uncertainties = [

np.array([0.05233423797,0.01812594243,0.001411105311,0.0006369746463,0.001
200025031])),

np.array([0.03365035905,0.00816091397,0.0005261995819,0.000321613588,0.000
1182116746])),

np.array([0.02061913609,0.00913591686,0.004509964107,0.003227424767,0.0019
24212176]))

]

labels = [
    r"OmniJet- $\alpha$  - Ideal",
    r"OmniJet- $\alpha$  - Controlled",
    r"OmniJet- $\alpha$  - From Scratch"
]

### Second Plot Data
y_values_2 = [
    np.array([0.751022,0.88742,0.970702,0.978844,0.982808])),
    np.array([0.711656,0.92549,0.971838,0.97901,0.982578])),
    np.array([0.598234,0.653992,0.82001,0.900622,0.935444]))

]

y_uncertainties_2 = [

np.array([0.04827149954,0.01162668805,0.0003308965095,0.0002309112384,0.00
0157138474])),

np.array([0.03398901856,0.004885441382,0.0003027457679,0.0001924837655,0.0
0007916754385])),

```

```

np.array([0.03505954767,0.01414145387,0.003085155993,0.0009017787977,0.000
4892289852])
]

labels_2 = [
    r"OmniJet- $\alpha$  - Ideal",
    r"OmniJet- $\alpha$  - Controlled",
    r"OmniJet- $\alpha$  - From Scratch"
]

def PlotWithUncertainty(ax, x, y_sets, y_errs, labels, ylabel,
title=None):
    colors = plt.cm.tab10.colors

    for i, (y, err, label) in enumerate(zip(y_sets, y_errs, labels)):
        color = colors[i % len(colors)]

        ax.plot(
            x,
            y,
            label=label,
            color=color,
            linewidth=2.5
        )

        ax.fill_between(
            x,
            y - err,
            y + err,
            color=color,
            alpha=0.2
        )

        ax.errorbar(

```

```

        x,
        y,
        yerr=err,
        fmt="none",
        color=color,
        capsize=3,
        alpha=0.8
    )

    ax.set_xlabel("Number of Training Jets")
    ax.set_ylabel(ylabel)
    ax.set_xscale('log')
    if title:
        ax.set_title(title)

    ax.grid(alpha=0.3)
    ax.legend()

if y_values_2 is None:
    fig, ax = plt.subplots(figsize=(7,5))
    PlotWithUncertainty(ax, x, y_values, y_uncertainties, labels, "Plot")
else:
    fig, axes = plt.subplots(1, 2, figsize=(12,5), sharey=False)

    PlotWithUncertainty(
        axes[0],
        x,
        y_values,
        y_uncertainties,
        labels,
        "Accuracy", ### Set Title and Y axis label
        "Accuracy Against Dataset Size"
    )

```

```

PlotWithUncertainty(
    axes[1],
    x,
    y_values_2,
    y_uncertainties_2,
    labels_2,
    "AUC",
    "AUC Against Dataset Size"
)

plt.tight_layout()
plt.savefig("OmniJetperformance.png", dpi=300, bbox_inches='tight')
plt.show()

```

Further Snippets:

There were a number of small modifications to other scripts. The most important ones are shown here. Firstly, these lines were added to the OmniLearn training script so it would save checkpoints to different file paths, again allowing for more rapid and simultaneous testing:

```

if hvd.rank() == 0:
    checkpoint_name = utils.get_model_name(flags, flags.fine_tune,

add_string="_{}".format(flags.nid) if flags.nid>0 else '')
    checkpoint_path = os.path.join(flags.folder, 'checkpoints',
datetime.datetime.now().strftime("%Y%m%d_%H%M%S")+"_"+checkpoint_name)

```

The following lines of code were added to OmniJet- α 's training to enable random seeding. This allowed for different results to be obtained on repeat tests to calculate uncertainties:

```

# set seed from random number generators
if cfg.get("seed") is None:
    cfg.seed = random.randint(0, 1_000_000)

```

```

if cfg.get("seed"):
    L.seed_everything(cfg.seed, workers=True)

```

The next snippets modify the OmniJet- α metrics for generation to use Wasserstein distance:

```

def quantiled_kl_divergence(sample_ref: np.ndarray, sample_approx:
np.ndarray, n_bins: int = 30):
    """Calculate the KL divergence using quantiles on sample_ref to define
the bounds.

Parameters
-----
sample_ref : np.ndarray
    The first sample to compare (this is the reference, so in the
context of
    jet generation, those are the real jets).
sample_approx : np.ndarray
    The second sample to compare (this is the model/approximation, so
in the
    context of jet generation, those are the generated jets).
n_bins : int
    The number of bins to use for the histogram. Those bins are
defined by
    equiprobably quantiles of sample_ref.
    """
    bins = np.quantile(sample_ref, np.linspace(0.001, 0.999, n_bins))
    hist_1 = np.histogram(sample_ref, bins, density=True)[0] + 1e-8
    hist_2 = np.histogram(sample_approx, bins, density=True)[0] + 1e-8
    return (scipy.stats.entropy(hist_1, hist_2),
scipy.stats.wasserstein_distance(hist_1, hist_2))

```


Appendix B: Meeting Minutes {BB}

Here are some notes from the main meetings we had with our advisor (and some additional ones of just us).

Date: 13/01/26

Attendees: AA, BB, CC, DD, EE, Mario Campanelli

Summary of Meeting:

- Advisor went through the structure of a foundational model, in particular going into the tokenisation and how classification and generation works.

Main Objectives:

- Understand the project scope and goals.
- Gain a basic understanding of foundational model architecture.
- Apply architectural concepts to the context of particle jets.
- Investigate how classification and generation works.
- Read through OmniJet- α paper and potentially have a look at the code.

Date: 6/02/26

Attendees: AA, BB, CC, DD, EE, Mario Campanelli

Summary of Meeting:

- Been given a username and password for GPU cluster.
- Go to the DIAS website and read the guide on how to submit slurm jobs.
- Also to login and use the cluster, need to be connected to UCL VPN or wifi (see UCL website for guidance on this: <https://www.ucl.ac.uk/isd/services/get-connected/ucl-virtual-private-network-vpn>).
- Need to be using the newest version of VS-code: v1.98.2 (may need to go back to older versions).
- See <https://uclphysast.github.io/clusters/dias/> to look at how to run the cluster.
- Guidance on slurm cheat sheets: <https://slurm.schedmd.com/pdfs/summary.pdf>
- This means we don't have to use docker as a virtual environment.
- Virtual storage is available which solves our issue of not enough storage to download the JetClass dataset.

Main Objectives:

- Access and be able to log into the GPU clusters.

Date: 10/02/26

Attendees: AA, BB, CC, DD, EE, Yuanda Zhu, Imran Rahman, Mario Campanelli

Summary of Meeting:

- Yuanda did a walk through on which directories to change.
- Went through how to run and view jobs we submitted.
- Going over how to run the tokenisation.
- Told us what to look for to see a successful job.

Main Objectives:

- Understand and be able to run the submit scripts written by Yuanda.
- Ensure everyone is connected to DIAS.
- Set up the .env folder.
- Ensure all the directories match to our own devices.

Date: 13/02/26

Attendees: AA, BB, CC, DD, EE, Mario Campanelli

Summary of Meeting:

- Mario had found another foundational model, similar to OmniJet- α called OmniLearn.
- He thought it would be interesting to discuss how the two worked: similarities and differences.

Main Objectives:

- Read through the OmniLearn paper - look into its architectural differences.
- Potentially look into running OmniLearn to also gather data to use to compare with OmniJet- α

Date: 17/02/26

Attendees: AA, BB, CC, DD, EE

Summary of Meeting:

- Split ourselves off into running OmniJet tests (AA, BB, DD) or OmniLearn tests (CC, EE) for performance.
- These tests included, dataset sizes, freezing and unfreezing the backbone, testing for different datasets other than the JetClass dataset, the ordering of the jets.
- Also divided research areas for each member.
- BB: Particle jet importance, methodologies for both models, advantages and disadvantages.
- EE: why foundational models are needed for particle physics and how best to compare the two models.
- DD: the importance of training on generation and why it's good for research.

Main Objectives:

- Run/research the assigned areas as above and summarise any findings

Date: 03/03/26

Attendees: AA, BB, CC, DD, EE, Mario Campanelli

Summary of Meeting:

- Discussed some issues we had with running each model:
 - OmniLearn kept running on CPU instead of GPU which was not suitable for running certain tasks.
 - Issues with tensor flow (software for machine learning).
- Presented our first results:
 - Jet tagging plots of its accuracy. These of which omnilearn converged to higher accuracies on smaller datasets compared to OmniJet.
 - OmniLearn uses particle classification (e.g. charge) and OmniJet does not.
 - OmniJet uses classification using momentum and 2 angles.
- Started to discuss the structure of a report and some other investigations to look into:
 - What happens when charge is taken away on OmniLearn? How does this compare to OmniJet?
 - Have a chapter about the differences between the two.
 - Chapter with direct comparisons (e.g. a test using the same datasets).
 - Potentially come up with ideas for future models and improvements for jet tagging and generation.

- OmniJet- α uses a fixed seed which means you get the same result each iteration. This needs to be randomised each run.

Main Objectives:

- Test the convergence speed for both models.
- Test how the data size of input jets affects accuracy.
- Run a peak accuracy test over different jet types.
- Test a frozen vs unfrozen backbone.
- Start writing the group report (mainly introductory and theory stuff).

Date: 16/03/26

Attendees: AA, BB, CC, DD, EE, Mario Campanelli

Summary of Meeting:

- First report draft is sent in and waiting for feedback.
- Asked for general guidance on the report, e.g word count and structure of it.
- Clarified how the poster session is going to work:
 - No need for a full on presentation in the standard sense.
 - Be prepared to answer questions.
 - Include key plots and schematics which portray how the models work.
- Some answers for clarification on the individual report:

- Very similar structure to the group report.
- Can reference our group report.

Main Objectives:

- Start and finish making the project poster.
- Make a start on Individual reports.

Date: 20/03/26

Attendees: AA, BB, CC, DD, EE, Mario Campanelli

Summary of Meeting:

- Given feedback on the group report:
 - Good overall but need to add in the appendices.
 - Change the title to properly fit the investigation.
 - Make sure the referencing is consistent.
 - More explanation on what foundational models are in general especially since they can be applied to areas outside of particle physics.
 - Need to include the final generation plots.
- Also decided on official role titles of each group member.

Main Objectives:

- Finalize the group report (making sure everything is consistent with each other and doing some final editing) and the individual report to be submitted.
- Finish off the poster to get it printed.